

Call-Site **this** versus Lexical **this**

JavaScript & TypeScript Essentials

Cloud Contraptions LLC - www.cloudcontraptions.com

Two Big Ideas

- Regular `function` → **call-site** `this` (how it's called)
- Arrow `=>` → **lexical** `this` (where it's defined)
- Choosing correctly avoids most `this` confusion

```
function showThis() { console.log(this); }

const obj = { showThis };
obj.showThis(); // this = obj

const ref = obj.showThis;
ref();          // this = global / undefined
```

Call-Site **this** Patterns

- `obj.method()` → `this` is `obj`
- Detached call → `this` is global / undefined

```
const user = {  
  name: "Alice",  
  sayHi: function () {  
    console.log("Hi, I am", this.name);  
  }  
};  
  
user.sayHi();           // "Hi, I am Alice"  
const greet = user.sayHi;  
greet();                // this.name is undefined
```

Ask the Call Site, Not the Definition

How it is called	<code>this</code> is
<code>new Fn()</code>	the brand-new object
<code>fn.call(x)</code> / <code>.apply</code> / <code>.bind(x)</code>	whatever you passed
<code>obj.method()</code>	<code>obj</code> , left of the dot
<code>fn()</code> bare	<code>undefined</code> (strict) / <code>globalThis</code>
any arrow function	the enclosing scope's <code>this</code>

- Work down the list; the first match wins

The Nested Function Problem

- An inner `function` has its **own** call-site `this`
- Inside `setTimeout` it is **not** the outer object

```
const counter = {  
  value: 0,  
  start: function () {  
    setTimeout(function () {  
      this.value++;           // this is NOT counter  
      console.log(this.value); // wrong  
    }, 1000);  
  }  
};  
  
counter.start();
```

Old Workarounds

- Save `this` in a variable, or `.bind(this)`
- Both work but are wordy and easy to forget

```
start: function () {  
  const self = this;           // capture  
  setTimeout(function () {  
    self.value++;  
  }, 1000);  
  
  setTimeout(function () {  
    this.value++;  
  }.bind(this), 1000);        // or bind  
}
```

Arrow Functions to the Rescue

- Arrows have **no own `this`**; they use the surrounding one
- No `self = this`, no `.bind` needed

```
const counter = {  
  value: 0,  
  start: function () {  
    setTimeout(() => {  
      this.value++;           // this is counter  
      console.log(this.value);  
    }, 1000);  
  }  
};  
  
counter.start();
```

Arrows in Array Callbacks

- Same fix inside `forEach`, `map`, event listeners
- Arrow reuses the method's `this`

```
const list = {  
  items: ["a", "b", "c"],  
  logAll: function () {  
    this.items.forEach((item) => {  
      console.log(this.items.length, item); // this is list  
    });  
  }  
};  
  
list.logAll();
```

Pitfall: Arrow as a Method

- An arrow **method** captures the outer (often global) `this`
- Use `function` to define methods; arrows for nested callbacks

```
const user = {  
  name: "Alice",  
  sayHi: () => {  
    console.log("Hi", this.name); // NOT user  
  }  
};  
  
user.sayHi(); // this.name is undefined
```

Class Arrow Fields Survive Detaching

- A class field holding an arrow captures `this` at construction

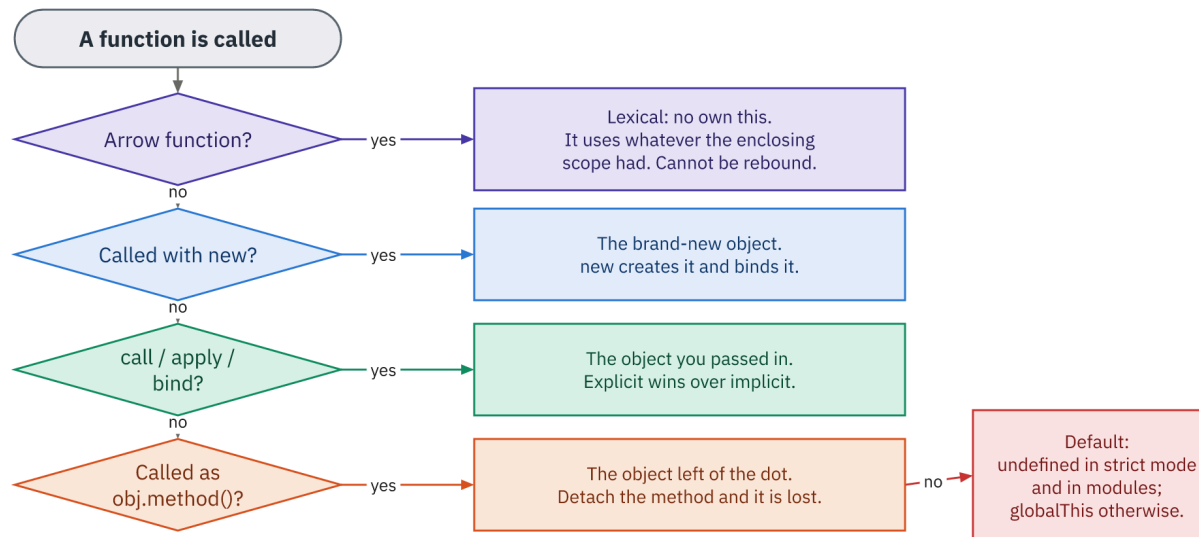
```
class Watch {  
  count = 0;  
  ring = () => {           // arrow FIELD, not a method  
    this.count++;  
    console.log(this.count);  
  };  
}  
  
const w = new Watch();  
const detached = w.ring;  
detached(); // 1 - still bound
```

- Use it for handlers you pass elsewhere, not for every method

The Whole Decision, on One Page

How this Is Decided

Ask the call site, not where the function was written. Work down: the first match wins.



- Work down the list: the first rule that matches wins
- Ask the **call site**, not where the function was written